

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport"
  content="width=device-width, initial-scale=1, viewport-fit=cover">
<meta name="apple-mobile-web-app-capable" content="yes">
<meta name="apple-mobile-web-app-status-bar-style" content="default">
<meta name="apple-mobile-web-app-title" content="LOOK Events">
<meta name="theme-color" content="#f2f2f7">
<meta name="format-detection" content="telephone=no">
<link rel="manifest" href="./manifest.json">
<link rel="apple-touch-icon" href="./icon.svg">
<title>LOOK Events</title>
<style>
:root {
  --page: #F2F2F7;
  --card: #FFFFFF;
  --blue: #007AFF;
  --red: #FF3B30;
  --secondary: #6C6C70;
  --separator: #E5E5EA;
  --walkin-bg: #FFF3CD;
  --walkin-text: #7D5A00;
  --toast: rgba(50, 50, 50, 0.92);
  --nav: rgba(242, 242, 247, 0.94);
  --text: #1C1C1E;
  --green: #248A3D;
  --shadow: 0 0.5px 0 rgba(0, 0, 0, 0.1);
  --font: -apple-system, BlinkMacSystemFont, 'SF Pro Text', 'Helvetica Neue', Ar
ial, sans-serif;
}

* {
  box-sizing: border-box;
}

html,
body {
  width: 100%;
  height: 100%;
  margin: 0;
  overflow: hidden;
  background: var(--page);
  color: var(--text);
  font-family: var(--font);
  -webkit-font-smoothing: antialiased;
  -webkit-tap-highlight-color: transparent;
}

body.sheet-open {
  overflow: hidden;
}

button,
input,
textarea {
  font: inherit;
}

button {
  border: 0;

```

```

    cursor: pointer;
}

button:disabled {
    cursor: default;
    opacity: 0.45;
}

button:active:not(:disabled),
.event-row:active,
.result-row.can-add:active,
.walkin-row:active {
    opacity: 0.5;
}

:focus-visible {
    outline: 3px solid rgba(0, 122, 255, 0.45);
    outline-offset: 2px;
}

#app {
    position: fixed;
    inset: 0;
    overflow: hidden;
    background: var(--page);
}

.screen {
    position: absolute;
    inset: 0;
    overflow-y: auto;
    background: var(--page);
    transform: translateX(100%);
    transition: transform 320ms cubic-bezier(0.25, 0.46, 0.45, 0.94);
    will-change: transform;
    -webkit-overflow-scrolling: touch;
}

.screen.active {
    transform: translateX(0);
}

.screen.behind {
    transform: translateX(-28%);
}

.nav {
    position: sticky;
    top: 0;
    z-index: 20;
    display: grid;
    grid-template-columns: minmax(90px, 1fr) auto minmax(90px, 1fr);
    align-items: end;
    min-height: calc(44px + env(safe-area-inset-top));
    padding: env(safe-area-inset-top) 10px 0;
    background: var(--nav);
    border-bottom: 0.5px solid rgba(0, 0, 0, 0.15);
    backdrop-filter: blur(20px) saturate(180%);
    -webkit-backdrop-filter: blur(20px) saturate(180%);
}

.nav-title {

```

```
display: flex;
align-items: center;
justify-content: center;
min-height: 44px;
max-width: 56vw;
overflow: hidden;
font-size: 17px;
font-weight: 600;
text-align: center;
text-overflow: ellipsis;
white-space: nowrap;
}

.nav-side {
display: flex;
min-height: 44px;
align-items: center;
}

.nav-side.right {
justify-content: flex-end;
}

.nav-btn {
min-width: 44px;
min-height: 44px;
padding: 0 8px;
border-radius: 11px;
background: transparent;
color: var(--blue);
font-size: 17px;
line-height: 1;
}

.home-head {
display: flex;
align-items: center;
justify-content: space-between;
padding: calc(18px + env(safe-area-inset-top)) 20px 14px;
}

.large-title {
margin: 0;
font-size: 34px;
font-weight: 700;
letter-spacing: 0;
}

.content {
padding: 18px 16px 28px;
}

.event-content {
padding-bottom: 132px;
}

.section-title {
margin: 24px 4px 8px;
color: var(--secondary);
font-size: 13px;
font-weight: 600;
letter-spacing: 0.04em;
}
```

```
    text-transform: uppercase;
}

.card {
    overflow: hidden;
    border-radius: 12px;
    background: var(--card);
    box-shadow: var(--shadow);
}

.cell {
    display: flex;
    min-height: 48px;
    align-items: center;
    gap: 12px;
    padding: 10px 16px;
    border-bottom: 0.5px solid var(--separator);
}

.cell:last-child {
    border-bottom: 0;
}

.event-row {
    display: grid;
    width: 100%;
    grid-template-columns: 1fr auto;
    align-items: center;
    border-radius: 0;
    background: var(--card);
    color: inherit;
    text-align: left;
}

.event-row-main {
    min-width: 0;
}

.event-name {
    overflow: hidden;
    font-size: 17px;
    font-weight: 600;
    text-overflow: ellipsis;
    white-space: nowrap;
}

.event-date,
.event-meta,
.muted {
    color: var(--secondary);
}

.event-date {
    margin-top: 3px;
    font-size: 14px;
}

.event-meta {
    margin-left: 14px;
    font-size: 13px;
    text-align: right;
    white-space: nowrap;
}
```

```

}

.empty {
  padding: 28px 18px;
  color: var(--secondary);
  text-align: center;
}

.empty strong {
  display: block;
  margin-bottom: 8px;
  color: var(--text);
  font-size: 20px;
}

.primary-btn,
.secondary-btn,
.destructive-btn {
  display: inline-flex;
  min-height: 50px;
  align-items: center;
  justify-content: center;
  border-radius: 14px;
  font-size: 17px;
  font-weight: 600;
}

.primary-btn {
  width: 100%;
  padding: 16px;
  background: var(--blue);
  color: #FFFFFF;
}

.secondary-btn {
  padding: 12px 16px;
  background: rgba(0, 122, 255, 0.11);
  color: var(--blue);
}

.destructive-btn {
  padding: 12px 16px;
  background: rgba(255, 59, 48, 0.11);
  color: var(--red);
}

.setup-form {
  display: grid;
  gap: 12px;
}

.field {
  display: grid;
  gap: 7px;
  margin-bottom: 12px;
}

.field label,
.field-label {
  color: var(--secondary);
  font-size: 13px;
  font-weight: 600;
}

```

```
}

.text-input,
.date-input,
.search-input,
.name-input,
.textarea {
  width: 100%;
  min-height: 48px;
  border: 0.5px solid rgba(0, 0, 0, 0.12);
  border-radius: 12px;
  background: #FFFFFF;
  color: var(--text);
  font-size: 17px;
}
```

```
.text-input,
.date-input,
.search-input,
.name-input {
  padding: 0 13px;
}
```

```
.textarea {
  min-height: 220px;
  padding: 12px 13px;
  resize: vertical;
}
```

```
.import-grid {
  display: grid;
  gap: 10px;
  padding: 12px;
}
```

```
.import-btn {
  display: flex;
  width: 100%;
  min-height: 52px;
  align-items: center;
  justify-content: space-between;
  padding: 0 14px;
  border-radius: 12px;
  background: #F7F7FA;
  color: var(--text);
  font-size: 16px;
  font-weight: 600;
  text-align: left;
}
```

```
.import-btn span:last-child {
  color: var(--blue);
  font-size: 20px;
  font-weight: 400;
}
```

```
.setup-error,
.ocr-error {
  display: none;
  margin: 10px 4px 0;
  color: var(--red);
  font-size: 14px;
}
```

```
    line-height: 1.35;
}

.setup-error.visible,
.ocr-error.visible {
    display: block;
}

.preview-head {
    display: flex;
    align-items: center;
    justify-content: space-between;
    margin: 24px 4px 8px;
}

.preview-count {
    color: var(--secondary);
    font-size: 13px;
    font-weight: 600;
    letter-spacing: 0.04em;
    text-transform: uppercase;
}

.clear-btn {
    min-width: 44px;
    min-height: 44px;
    padding: 0 8px;
    border-radius: 11px;
    background: transparent;
    color: var(--red);
    font-size: 15px;
}

.preview-list {
    max-height: 310px;
    overflow-y: auto;
    -webkit-overflow-scrolling: touch;
}

.preview-row {
    display: grid;
    grid-template-columns: 1fr auto;
    gap: 8px;
    align-items: center;
    padding: 8px 12px;
    border-bottom: 0.5px solid var(--separator);
}

.preview-row:last-child {
    border-bottom: 0;
}

.small-btn {
    min-width: 44px;
    min-height: 44px;
    padding: 0 11px;
    border-radius: 11px;
    background: #F7F7FA;
    color: var(--blue);
    font-size: 15px;
    font-weight: 600;
}
```

```
.small-btn.red {
  color: var(--red);
}

.stats {
  position: sticky;
  top: calc(44px + env(safe-area-inset-top));
  z-index: 12;
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 8px;
  padding: 10px 16px;
  background: rgba(242, 242, 247, 0.88);
  border-bottom: 0.5px solid rgba(0, 0, 0, 0.08);
  backdrop-filter: blur(16px);
  -webkit-backdrop-filter: blur(16px);
}

.stat {
  min-width: 0;
  padding: 8px 6px;
  border-radius: 12px;
  background: rgba(255, 255, 255, 0.74);
  text-align: center;
}

.stat-value {
  font-size: 20px;
  font-weight: 700;
}

.stat-label {
  margin-top: 2px;
  color: var(--secondary);
  font-size: 11px;
  font-weight: 600;
  text-transform: uppercase;
}

.groups {
  display: grid;
  gap: 14px;
  padding: 16px 16px calc(160px + env(safe-area-inset-bottom));
}

.group-card {
  overflow: hidden;
  border-radius: 12px;
  background: var(--card);
  box-shadow: var(--shadow);
}

.group-head {
  display: grid;
  grid-template-columns: auto auto 1fr auto;
  gap: 10px;
  align-items: center;
  padding: 12px 12px 10px;
  border-bottom: 0.5px solid var(--separator);
}
```



```
.reorder-controls {
  display: grid;
  gap: 2px;
}

.reorder-btn {
  display: grid;
  width: 28px;
  height: 22px;
  min-width: 28px;
  min-height: 22px;
  place-items: center;
  border-radius: 8px;
  background: transparent;
  color: var(--secondary);
  font-size: 13px;
  line-height: 1;
}

.reorder-btn:disabled {
  color: rgba(108, 108, 112, 0.32);
  opacity: 1;
}

.number-circle {
  display: grid;
  width: 36px;
  height: 36px;
  place-items: center;
  border-radius: 50%;
  background: var(--blue);
  color: #FFFFFF;
  font-size: 17px;
  font-weight: 700;
}

.group-label {
  width: 100%;
  min-width: 0;
  min-height: 44px;
  border: 0;
  border-radius: 10px;
  background: #F7F7FA;
  color: var(--text);
  font-size: 17px;
  font-weight: 600;
  padding: 0 10px;
}

.confirm-actions {
  display: flex;
  gap: 8px;
  align-items: center;
}

.people {
  display: grid;
}

.person-row {
  display: grid;
  grid-template-columns: auto minmax(0, 1fr) auto;
```

```

    gap: 10px;
    align-items: center;
    min-height: 58px;
    padding: 9px 12px;
    border-bottom: 0.5px solid var(--separator);
}

.person-row:last-child {
    border-bottom: 0;
}

.person-main {
    min-width: 0;
}

.person-name-line {
    display: flex;
    min-width: 0;
    align-items: center;
    gap: 8px;
}

.person-name {
    overflow: hidden;
    font-size: 17px;
    font-weight: 500;
    text-overflow: ellipsis;
    white-space: nowrap;
}

.avatar {
    display: grid;
    width: 34px;
    height: 34px;
    flex: 0 0 auto;
    place-items: center;
    border-radius: 50%;
    background: #D9E8FF;
    color: #004C9E;
    font-size: 12px;
    font-weight: 700;
}

.avatar.alt-1 { background: #E9F7EF; color: #1D6B35; }
.avatar.alt-2 { background: #FEE9E7; color: #A7271E; }
.avatar.alt-3 { background: #FFF1D6; color: #805100; }
.avatar.alt-4 { background: #EDE8FF; color: #4D32A8; }

.walkin-badge {
    display: inline-flex;
    align-items: center;
    min-height: 18px;
    padding: 2px 6px;
    border-radius: 20px;
    background: var(--walkin-bg);
    color: var(--walkin-text);
    font-size: 10px;
    font-weight: 600;
    text-transform: uppercase;
    white-space: nowrap;
}

```

```

.person-actions {
  display: flex;
  gap: 4px;
  align-items: center;
}

.inline-edit {
  width: 100%;
  min-height: 44px;
  border: 0.5px solid rgba(0, 122, 255, 0.5);
  border-radius: 10px;
  padding: 0 10px;
  background: #FFFFFF;
  font-size: 17px;
}

.group-foot {
  padding: 10px 12px 12px;
}

.add-person-btn {
  width: 100%;
  min-height: 48px;
  border-radius: 12px;
  background: rgba(0, 122, 255, 0.11);
  color: var(--blue);
  font-size: 16px;
  font-weight: 600;
}

.fab {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  position: fixed;
  right: 24px;
  bottom: calc(24px + env(safe-area-inset-bottom));
  z-index: 100;
  min-height: 54px;
  max-width: calc(100vw - 48px);
  padding: 0 22px;
  border-radius: 999px;
  background: var(--blue);
  box-shadow: 0 4px 20px rgba(0, 122, 255, 0.42);
  color: #FFFFFF;
  font-size: 17px;
  font-weight: 700;
  padding-bottom: env(safe-area-inset-bottom);
}

.fab[hidden] {
  display: none;
}

.export-copy {
  display: block;
  margin: 4px 4px 18px;
  color: var(--secondary);
  font-size: 15px;
  line-height: 1.4;
}

```

```

.export-preview {
  max-height: 52vh;
  overflow: auto;
  margin-bottom: 16px;
  padding: 14px;
  border-radius: 12px;
  background: #1C1C1E;
  color: #FFFFFF;
  font-family: ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas, monospace;
  font-size: 13px;
  line-height: 1.5;
  white-space: pre;
  -webkit-overflow-scrolling: touch;
}

.sheet-overlay {
  position: fixed;
  inset: 0;
  z-index: 80;
  background: rgba(0, 0, 0, 0.28);
  opacity: 0;
  transition: opacity 220ms ease;
}

.sheet-overlay.open {
  opacity: 1;
}

.sheet-overlay[hidden],
.sheet[hidden] {
  display: none;
}

.sheet {
  position: fixed;
  left: 0;
  right: 0;
  bottom: 0;
  z-index: 90;
  max-height: min(84vh, 760px);
  overflow: hidden;
  padding-bottom: env(safe-area-inset-bottom);
  border-radius: 22px 22px 0 0;
  background: var(--page);
  box-shadow: 0 -12px 36px rgba(0, 0, 0, 0.22);
  transform: translateY(110%);
  transition: transform 400ms cubic-bezier(0.34, 1.56, 0.64, 1);
  will-change: transform;
}

.sheet.open {
  transform: translateY(0);
}

.sheet-head {
  display: grid;
  grid-template-columns: minmax(70px, 1fr) auto minmax(70px, 1fr);
  align-items: center;
  min-height: 56px;
  padding: 8px 12px 6px;
  border-bottom: 0.5px solid rgba(0, 0, 0, 0.12);
}

```

```
.sheet-title {
  min-width: 0;
  overflow: hidden;
  font-size: 17px;
  font-weight: 700;
  text-align: center;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.sheet-body {
  max-height: calc(min(84vh, 760px) - 56px - env(safe-area-inset-bottom));
  overflow-y: auto;
  padding: 14px 16px 18px;
  -webkit-overflow-scrolling: touch;
}

.results {
  display: grid;
  gap: 8px;
  margin-top: 12px;
}

.result-row,
.walkin-row {
  display: grid;
  grid-template-columns: auto minmax(0, 1fr) auto;
  gap: 10px;
  align-items: center;
  min-height: 56px;
  padding: 10px 12px;
  border-radius: 12px;
  background: #FFFFFF;
  box-shadow: var(--shadow);
  color: inherit;
  text-align: left;
}

.result-name {
  overflow: hidden;
  font-size: 17px;
  font-weight: 600;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.result-sub {
  margin-top: 2px;
  color: var(--secondary);
  font-size: 12px;
}

.plus-btn {
  width: 44px;
  height: 44px;
  border-radius: 50%;
  background: var(--blue);
  color: #FFFFFF;
  font-size: 28px;
  line-height: 1;
}
```

```

.in-group {
  color: var(--secondary);
  font-size: 14px;
  font-weight: 600;
  white-space: nowrap;
}

.sheet-actions {
  display: grid;
  gap: 10px;
  margin-top: 14px;
}

.ocr-status {
  display: grid;
  gap: 16px;
  justify-items: center;
  padding: 30px 10px;
  color: var(--secondary);
  text-align: center;
}

.spinner {
  width: 34px;
  height: 34px;
  border: 3px solid rgba(0, 122, 255, 0.18);
  border-top-color: var(--blue);
  border-radius: 50%;
  animation: spin 0.9s linear infinite;
}

@keyframes spin {
  to { transform: rotate(360deg); }
}

.ocr-list {
  display: grid;
  gap: 8px;
}

.ocr-row {
  display: grid;
  grid-template-columns: 1fr auto;
  gap: 8px;
  align-items: center;
}

.toast {
  position: fixed;
  left: 50%;
  bottom: calc(92px + env(safe-area-inset-bottom));
  z-index: 120;
  max-width: min(460px, calc(100vw - 32px));
  padding: 11px 16px;
  border-radius: 999px;
  background: var(--toast);
  color: #FFFFFF;
  font-size: 15px;
  font-weight: 600;
  line-height: 1.25;
  text-align: center;
}

```

```

    opacity: 0;
    transform: translate(-50%, 12px);
    transition: opacity 180ms ease, transform 180ms ease;
}

.toast.show {
    opacity: 1;
    transform: translate(-50%, 0);
}

.visually-hidden {
    position: absolute;
    width: 1px;
    height: 1px;
    overflow: hidden;
    clip: rect(0 0 0 0);
    white-space: nowrap;
}

@media (min-width: 700px) {
    .content,
    .groups,
    .event-content {
        max-width: 760px;
        margin: 0 auto;
    }

    .sheet {
        left: 50%;
        right: auto;
        width: min(720px, 100vw);
        transform: translate(-50%, 110%);
    }

    .sheet.open {
        transform: translate(-50%, 0);
    }
}
</style>
<script defer src="https://cdn.jsdelivr.net/npm/tesseract.js@4/dist/tesseract.min.js"></script>
<script defer src="https://cdn.jsdelivr.net/npm/xlsx@0.18.5/dist/xlsx.full.min.js"></script>
</head>
<body>
<div id="app">
    <section id="s-home" class="screen active" aria-labelledby="home-title">
        <div class="home-head">
            <h1 id="home-title" class="large-title">Events</h1>
            <button id="home-new" class="nav-btn" type="button">New</button>
        </div>
        <main class="content">
            <div id="home-list"></div>
        </main>
    </section>

    <section id="s-setup" class="screen" aria-labelledby="setup-title">
        <header class="nav">
            <div class="nav-side">
                <button id="setup-back" class="nav-btn" type="button">< Back</button>
            </div>
            <div id="setup-title" class="nav-title">New Event</div>
        </header>
    </section>
</div>

```

```

    <div class="nav-side right"></div>
</header>
<main class="content">
  <div class="setup-form">
    <div class="field">
      <label for="event-name">Event name</label>
      <input id="event-name" class="text-input" type="text" autocomplete="of
f" required>
    </div>
    <div class="field">
      <label for="event-date">Date</label>
      <input id="event-date" class="date-input" type="date">
    </div>
  </div>

  <h2 class="section-title">Import Alpha Guest List</h2>
  <div class="card import-grid">
    <button id="take-photo-btn" class="import-btn" type="button">
      <span>Take Photo of List</span>
      <span aria-hidden="true">></span>
    </button>
    <button id="choose-photo-btn" class="import-btn" type="button">
      <span>Choose Photo of List</span>
      <span aria-hidden="true">></span>
    </button>
    <button id="file-import-btn" class="import-btn" type="button">
      <span>Import Excel or CSV</span>
      <span aria-hidden="true">></span>
    </button>
    <button id="paste-open-btn" class="import-btn" type="button">
      <span>Type or Paste Names</span>
      <span aria-hidden="true">></span>
    </button>
  </div>
  <p id="setup-error" class="setup-error" role="alert"></p>

  <div id="preview-wrap" hidden>
    <div class="preview-head">
      <div id="preview-count" class="preview-count">0 names loaded</div>
      <button id="clear-alpha-btn" class="clear-btn" type="button">Clear</bu
tton>
    </div>
    <div id="alpha-preview" class="card preview-list"></div>
  </div>

  <div style="height: 18px;"></div>
  <button id="start-event-btn" class="primary-btn" type="button" disabled>St
art Event</button>
</main>
</section>

<section id="s-event" class="screen" aria-labelledby="event-title">
  <header class="nav">
    <div class="nav-side">
      <button id="event-back" class="nav-btn" type="button">< Events</button>
    </div>
    <div id="event-title" class="nav-title">Event</div>
    <div class="nav-side right">
      <button id="event-export" class="nav-btn" type="button">Export</button>
    </div>
  </header>
  <div class="stats" aria-label="Event stats">

```



```

<div class="stat">
  <div id="stat-groups" class="stat-value">0</div>
  <div class="stat-label">Groups</div>
</div>
<div class="stat">
  <div id="stat-people" class="stat-value">0</div>
  <div class="stat-label">People</div>
</div>
<div class="stat">
  <div id="stat-walkins" class="stat-value">0</div>
  <div class="stat-label">Walk-ins</div>
</div>
<div class="stat">
  <div id="stat-onlist" class="stat-value">0</div>
  <div class="stat-label">On List</div>
</div>
</div>
<main id="groups-container" class="groups"></main>
</section>

<section id="s-export" class="screen" aria-labelledby="export-title">
  <header class="nav">
    <div class="nav-side">
      <button id="export-back" class="nav-btn" type="button">< Back</button>
    </div>
    <div id="export-title" class="nav-title">Export</div>
    <div class="nav-side right">
      <button id="copy-export-btn" class="nav-btn" type="button">Copy</button>
    </div>
  </header>
  <main class="content">
    <p class="export-copy">Copy or download this tab-separated file for Excel.
  </p>
    <pre id="export-preview" class="export-preview"></pre>
    <button id="download-export-btn" class="primary-btn" type="button">Download as .txt</button>
  </main>
</section>
</div>

<button id="new-group-btn" class="fab" type="button" hidden>+ New Photo Group</button>

<div id="sheet-overlay" class="sheet-overlay" hidden></div>

<section id="sheet-search" class="sheet" role="dialog" aria-modal="true" aria-labelledby="search-title" hidden>
  <header class="sheet-head">
    <div></div>
    <div id="search-title" class="sheet-title">Add to Group</div>
    <div class="nav-side right">
      <button id="search-done" class="nav-btn" type="button">Done</button>
    </div>
  </header>
  <div class="sheet-body">
    <label class="visually-hidden" for="person-search">Search names</label>
    <input id="person-search" class="search-input" type="search" autocomplete="off">
    <div id="search-results" class="results"></div>
  </div>
</section>

```

```

<section id="sheet-ocr" class="sheet" role="dialog" aria-modal="true" aria-label="ocr-title" hidden>
  <header class="sheet-head">
    <div class="nav-side">
      <button id="ocr-cancel" class="nav-btn" type="button">Cancel</button>
    </div>
    <div id="ocr-title" class="sheet-title">Reading image...</div>
    <div></div>
  </header>
  <div class="sheet-body">
    <div id="ocr-processing" class="ocr-status">
      <div class="spinner" aria-hidden="true"></div>
      <div id="ocr-progress">Loading OCR engine...</div>
    </div>
    <p id="ocr-error" class="ocr-error" role="alert"></p>
    <div id="ocr-review" hidden>
      <div id="ocr-list" class="ocr-list"></div>
      <div class="sheet-actions">
        <button id="ocr-use" class="primary-btn" type="button" disabled>Use These Names</button>
      </div>
    </div>
  </div>
</section>

```

```

<section id="sheet-paste" class="sheet" role="dialog" aria-modal="true" aria-label="paste-title" hidden>
  <header class="sheet-head">
    <div class="nav-side">
      <button id="paste-cancel" class="nav-btn" type="button">Cancel</button>
    </div>
    <div id="paste-title" class="sheet-title">Type or Paste Names</div>
    <div></div>
  </header>
  <div class="sheet-body">
    <label class="visually-hidden" for="paste-text">Names</label>
    <textarea id="paste-text" class="textarea"></textarea>
    <p class="muted">One name per line. Example: Jane Smith</p>
    <div class="sheet-actions">
      <button id="paste-add" class="primary-btn" type="button">Add Names</button>
    </div>
  </div>
</section>

```

```

<div id="toast" class="toast" role="status" aria-live="polite" hidden></div>

<input id="camera-input" class="visually-hidden" type="file" accept="image/*" capture="environment" aria-label="Take photo of list">
<input id="photo-input" class="visually-hidden" type="file" accept="image/*" aria-label="Choose photo of list">
<input id="file-input" class="visually-hidden" type="file" accept=".xlsx,.xls,.csv,.txt" aria-label="Import Excel or CSV">

```

```

<script>
'use strict';

const STORE_KEY = 'look_v3';
const TESSERACT_URL = 'https://cdn.jsdelivr.net/npm/tesseract.js@4/dist/tesseract.min.js';
const XLSX_URL = 'https://cdn.jsdelivr.net/npm/xlsx@0.18.5/dist/xlsx.full.min.js';

```

```

let S = { events: [], activeId: null };
let stack = ['s-home'];
let setupAlphaList = [];
let currentSheet = null;
let searchGroupId = null;
let toastTimer = null;
let pendingGroupDeleteId = null;
let pendingPersonRemoveId = null;
let editingGroupId = null;
let editingPersonId = null;
let editOriginalName = '';
let activeOCRWorker = null;
let ocrBusy = false;
let ocrCancelled = false;
let ocrNames = [];
let ocrSession = 0;

function $(id) {
  return document.getElementById(id);
}

function push(id) {
  const curId = stack[stack.length - 1];
  const curEl = document.getElementById(curId);
  const nxtEl = document.getElementById(id);

  curEl.classList.remove('active');
  curEl.classList.add('behind');

  nxtEl.classList.remove('behind');
  nxtEl.classList.add('active');
  nxtEl.scrollTop = 0;

  stack.push(id);
  updateFabVisibility();
}

function pop() {
  if (stack.length <= 1) return;

  const curId = stack.pop();
  const prevId = stack[stack.length - 1];

  const curEl = document.getElementById(curId);
  const prevEl = document.getElementById(prevId);

  curEl.classList.remove('active');
  curEl.classList.remove('behind');

  prevEl.classList.remove('behind');
  prevEl.classList.add('active');
  updateFabVisibility();
}

function resetTo(id) {
  document.querySelectorAll('.screen').forEach(s => {
    s.classList.remove('active', 'behind');
  });

  document.getElementById(id).classList.add('active');
  stack = [id];
}

```

```

updateFabVisibility();
}

function updateFabVisibility() {
  const fab = $('new-group-btn');
  if (!fab) return;
  fab.hidden = stack[stack.length - 1] !== 's-event';
}

document.addEventListener('DOMContentLoaded', () => {
  loadState();
  prepareSetup();
  bindUI();
  renderHome();
  resetTo('s-home');
});

function bindUI() {
  $('home-new').addEventListener('click', () => {
    prepareSetup();
    push('s-setup');
    setTimeout(() => $('event-name').focus(), 340);
  });
  $('setup-back').addEventListener('click', () => pop());
  $('event-name').addEventListener('input', updateStartButton);
  $('start-event-btn').addEventListener('click', createEvent);
  $('clear-alpha-btn').addEventListener('click', () => {
    setupAlphaList = [];
    renderAlphaPreview();
    updateStartButton();
  });
  $('take-photo-btn').addEventListener('click', () => $('camera-input').click());
  ;
  $('choose-photo-btn').addEventListener('click', () => $('photo-input').click());
);
  $('file-import-btn').addEventListener('click', () => $('file-input').click());
  $('paste-open-btn').addEventListener('click', openPasteSheet);
  $('camera-input').addEventListener('change', event => handleImageInput(event.target));
  $('photo-input').addEventListener('change', event => handleImageInput(event.target));
  $('file-input').addEventListener('change', event => handleFileInput(event.target));
);
  $('event-back').addEventListener('click', () => {
    renderHome();
    pop();
  });
  $('event-export').addEventListener('click', openExportScreen);
  $('new-group-btn').addEventListener('click', createNewGroup);
  $('export-back').addEventListener('click', () => pop());
  $('copy-export-btn').addEventListener('click', copyExport);
  $('download-export-btn').addEventListener('click', downloadExport);
  $('sheet-overlay').addEventListener('click', () => {
    if (currentSheet === 'ocr' && ocrBusy) return;
    closeSheet();
  });
  $('search-done').addEventListener('click', closeSheet);
  $('person-search').addEventListener('input', renderSearchResults);
  $('person-search').addEventListener('keydown', handleSearchKeydown);
  $('paste-cancel').addEventListener('click', closeSheet);
  $('paste-add').addEventListener('click', addPastedNames);
  $('ocr-cancel').addEventListener('click', closeSheet);

```

```

$('ocr-use').addEventListener('click', useOCRNames);
document.addEventListener('keydown', event => {
  if (event.key === 'Escape' && currentSheet) {
    if (currentSheet === 'ocr' && ocrBusy) return;
    closeSheet();
  }
});
}

function uid() {
  if (window.crypto && typeof window.crypto.randomUUID === 'function') {
    return window.crypto.randomUUID();
  }
  return 'id-' + Date.now().toString(36) + '-' + Math.random().toString(36).slice(2, 10);
}

function todayISO() {
  const d = new Date();
  const tz = d.getTimezoneOffset() * 60000;
  return new Date(d.getTime() - tz).toISOString().slice(0, 10);
}

function loadState() {
  try {
    const raw = localStorage.getItem(STORE_KEY);
    if (!raw) {
      S = { events: [], activeId: null };
      return;
    }
    const parsed = JSON.parse(raw);
    S = {
      events: Array.isArray(parsed.events) ? parsed.events : [],
      activeId: parsed.activeId || null
    };
  } catch (error) {
    S = { events: [], activeId: null };
    showToast('Could not read saved data. Starting fresh.');
```

```

        toast.classList.remove('show');
        toastTimer = setTimeout(() => {
            toast.hidden = true;
            toastTimer = null;
        }, 200);
    }, 2400);
}
}

function showSetupError(message) {
    const el = $('setup-error');
    el.textContent = message;
    el.classList.add('visible');
}

function clearSetupError() {
    const el = $('setup-error');
    el.textContent = '';
    el.classList.remove('visible');
}

function prepareSetup() {
    setupAlphaList = [];
    $('event-name').value = '';
    $('event-date').value = todayISO();
    clearSetupError();
    renderAlphaPreview();
    updateStartButton();
}

function renderHome() {
    const list = $('home-list');
    list.textContent = '';
    if (!S.events.length) {
        const empty = document.createElement('div');
        empty.className = 'card empty';
        const title = document.createElement('strong');
        title.textContent = 'No events yet';
        const copy = document.createElement('div');
        copy.textContent = 'Create an event to begin tracking photo groups.';
        const spacer = document.createElement('div');
        spacer.style.height = '16px';
        const btn = document.createElement('button');
        btn.className = 'primary-btn';
        btn.type = 'button';
        btn.textContent = 'New Event';
        btn.addEventListener('click', () => {
            prepareSetup();
            push('s-setup');
            setTimeout(() => $('event-name').focus(), 340);
        });
        empty.append(title, copy, spacer, btn);
        list.append(empty);
        return;
    }

    const card = document.createElement('div');
    card.className = 'card';
    S.events
        .slice()
        .sort((a, b) => b.createdAt - a.createdAt)
        .forEach(event => {

```

```

const stats = eventStats(event);
const row = document.createElement('button');
row.type = 'button';
row.className = 'cell event-row';
row.addEventListener('click', () => {
  S.activeId = event.id;
  saveState();
  renderEventScreen();
  push('s-event');
});

const main = document.createElement('div');
main.className = 'event-row-main';
const name = document.createElement('div');
name.className = 'event-name';
name.textContent = event.name;
const date = document.createElement('div');
date.className = 'event-date';
date.textContent = event.date || '';
main.append(name, date);

const meta = document.createElement('div');
meta.className = 'event-meta';
meta.textContent = `${stats.groups} groups · ${stats.people} people · ${stats.walkins} walk-ins`;
row.append(main, meta);
card.append(row);
});
list.append(card);
}

function eventStats(event) {
  const groups = Array.isArray(event.groups) ? event.groups : [];
  const people = groups.reduce((sum, group) => sum + group.people.length, 0);
  const walkins = groups.reduce((sum, group) => sum + group.people.filter(person => person.walkin).length, 0);
  return {
    groups: groups.length,
    people,
    walkins,
    onList: Array.isArray(event.alphaList) ? event.alphaList.length : 0
  };
}

function updateStartButton() {
  const nameReady = $('event-name').value.trim().length > 0;
  $('start-event-btn').disabled = !(nameReady && setupAlphaList.length > 0);
}

function renderAlphaPreview() {
  const wrap = $('preview-wrap');
  const list = $('alpha-preview');
  const count = $('preview-count');
  list.textContent = '';
  count.textContent = `${setupAlphaList.length} ${setupAlphaList.length === 1 ? 'name' : 'names'} loaded`;
  wrap.hidden = setupAlphaList.length === 0;

  setupAlphaList.forEach((nameObj, index) => {
    const row = document.createElement('div');
    row.className = 'preview-row';
    const input = document.createElement('input');

```

```

input.className = 'name-input';
input.type = 'text';
input.value = nameObj.display;
input.setAttribute('aria-label', `Name ${index + 1}`);
input.addEventListener('keydown', event => {
  if (event.key === 'Enter') input.blur();
});
input.addEventListener('blur', () => {
  const normalized = normalizeName(input.value);
  if (!normalized) {
    setupAlphaList.splice(index, 1);
  } else {
    setupAlphaList[index] = nameFromDisplay(normalized);
  }
  setupAlphaList = sortAndDedupeNames(setupAlphaList);
  renderAlphaPreview();
  updateStartButton();
});

const remove = document.createElement('button');
remove.className = 'small-btn red';
remove.type = 'button';
remove.textContent = 'Delete';
remove.addEventListener('click', () => {
  setupAlphaList.splice(index, 1);
  renderAlphaPreview();
  updateStartButton();
});
row.append(input, remove);
list.append(row);
}

function setSetupAlphaList(names) {
  setupAlphaList = sortAndDedupeNames(names);
  renderAlphaPreview();
  updateStartButton();
}

function createEvent() {
  const eventName = $('event-name').value.trim();
  if (!eventName || setupAlphaList.length < 1) return;

  const now = Date.now();
  const event = {
    id: uid(),
    name: eventName,
    date: $('event-date').value || todayISO(),
    alphaList: setupAlphaList.map(name => ({ first: name.first, last: name.last,
display: name.display })),
    groups: [],
    createdAt: now,
    updatedAt: now
  };

  S.events.push(event);
  saveState();
  S.activeId = event.id;
  saveState();
  renderEventScreen();
  pop();
  push('s-event');

```



```

    renderHome();
}

function currentEvent() {
    return S.events.find(event => event.id === S.activeId) || null;
}

function touchEvent(event) {
    event.updatedAt = Date.now();
    saveState();
}

function renderEventScreen() {
    const event = currentEvent();
    if (!event) {
        renderHome();
        resetTo('s-home');
        return;
    }
    $('event-title').textContent = event.name;
    const stats = eventStats(event);
    $('stat-groups').textContent = stats.groups;
    $('stat-people').textContent = stats.people;
    $('stat-walkins').textContent = stats.walkins;
    $('stat-onlist').textContent = stats.onList;

    const container = $('groups-container');
    container.textContent = '';
    if (!event.groups.length) {
        const empty = document.createElement('div');
        empty.className = 'card empty';
        const title = document.createElement('strong');
        title.textContent = 'No photo groups yet';
        const copy = document.createElement('div');
        copy.textContent = 'Tap New Photo Group when the first group is photographed';
        empty.appendChild(title, copy);
        container.appendChild(empty);
        return;
    }

    event.groups.forEach((group, index) => container.appendChild(renderGroupCard(event, group, index)));
}

function renderGroupCard(event, group, groupIndex) {
    const card = document.createElement('article');
    card.className = 'group-card';
    card.id = `group-${group.id}`;

    const head = document.createElement('div');
    head.className = 'group-head';

    const number = document.createElement('div');
    number.className = 'number-circle';
    number.textContent = group.number;

    const reorder = document.createElement('div');
    reorder.className = 'reorder-controls';
    const moveUp = document.createElement('button');
    moveUp.className = 'reorder-btn';
    moveUp.type = 'button';

```

```

    moveUp.textContent = '▲';
    moveUp.setAttribute('aria-label', `Move ${group.label} || `Photo Group ${group.
number}`}` up`);
    moveUp.disabled = groupIndex === 0;
    moveUp.addEventListener('click', () => moveGroup(event, group.id, -1));
    const moveDown = document.createElement('button');
    moveDown.className = 'reorder-btn';
    moveDown.type = 'button';
    moveDown.textContent = '▼';
    moveDown.setAttribute('aria-label', `Move ${group.label} || `Photo Group ${grou
p.number}`}` down`);
    moveDown.disabled = groupIndex === event.groups.length - 1;
    moveDown.addEventListener('click', () => moveGroup(event, group.id, 1));
    reorder.append(moveUp, moveDown);

    const label = document.createElement('input');
    label.className = 'group-label';
    label.type = 'text';
    label.value = group.label || `Photo Group ${group.number}`;
    label.setAttribute('aria-label', `Label for photo group ${group.number}`);
    label.addEventListener('keydown', keyEvent => {
      if (keyEvent.key === 'Enter') label.blur();
    });
    label.addEventListener('blur', () => {
      const nextLabel = label.value.trim() || `Photo Group ${group.number}`;
      if (nextLabel !== group.label) {
        group.label = nextLabel;
        touchEvent(event);
        if (searchGroupId === group.id) renderSearchSheet();
      }
      label.value = group.label;
    });

    const deleteWrap = document.createElement('div');
    if (pendingGroupDeleteId === group.id) {
      deleteWrap.className = 'confirm-actions';
      const confirm = document.createElement('button');
      confirm.className = 'small-btn red';
      confirm.type = 'button';
      confirm.textContent = 'Confirm delete';
      confirm.addEventListener('click', () => {
        event.groups = event.groups.filter(item => item.id !== group.id);
        pendingGroupDeleteId = null;
        if (searchGroupId === group.id) closeSheet();
        touchEvent(event);
        renderEventScreen();
      });
      const cancel = document.createElement('button');
      cancel.className = 'small-btn';
      cancel.type = 'button';
      cancel.textContent = 'Cancel';
      cancel.addEventListener('click', () => {
        pendingGroupDeleteId = null;
        renderEventScreen();
      });
      deleteWrap.append(confirm, cancel);
    } else {
      const del = document.createElement('button');
      del.className = 'small-btn red';
      del.type = 'button';
      del.textContent = 'Delete';
      del.addEventListener('click', () => {

```

```

        pendingGroupDeleteId = group.id;
        pendingPersonRemoveId = null;
        renderEventScreen();
    });
    deleteWrap.append(del);
}

head.append(number, reorder, label, deleteWrap);
card.append(head);

const people = document.createElement('div');
people.className = 'people';
if (!group.people.length) {
    const empty = document.createElement('div');
    empty.className = 'cell muted';
    empty.textContent = 'No people added yet';
    people.append(empty);
} else {
    group.people.forEach((person, index) => people.append(renderPersonRow(event,
group, person, index)));
}
card.append(people);

const foot = document.createElement('div');
foot.className = 'group-foot';
const add = document.createElement('button');
add.className = 'add-person-btn';
add.type = 'button';
add.textContent = 'Add Person';
add.addEventListener('click', () => openSearchSheet(group.id));
foot.append(add);
card.append(foot);

return card;
}

function moveGroup(event, groupId, direction) {
    const index = event.groups.findIndex(group => group.id === groupId);
    const nextIndex = index + direction;
    if (index < 0 || nextIndex < 0 || nextIndex >= event.groups.length) return;
    [event.groups[index], event.groups[nextIndex]] = [event.groups[nextIndex], eve
nt.groups[index]];
    pendingGroupDeleteId = null;
    pendingPersonRemoveId = null;
    touchEvent(event);
    renderEventScreen();
}

function renderPersonRow(event, group, person, index) {
    const row = document.createElement('div');
    row.className = 'person-row';

    const avatar = document.createElement('div');
    avatar.className = `avatar alt-${index % 5}`;
    avatar.textContent = initials(person.name);

    const main = document.createElement('div');
    main.className = 'person-main';

    if (editingGroupId === group.id && editingPersonId === person.id) {
        const input = document.createElement('input');
        input.className = 'inline-edit';
    }

```

```

input.type = 'text';
input.value = person.name;
input.setAttribute('aria-label', `Edit ${person.name}`);
input.addEventListener('keydown', keyEvent => {
  if (keyEvent.key === 'Enter') {
    keyEvent.preventDefault();
    commitPersonEdit(event, group, person, input.value);
  }
  if (keyEvent.key === 'Escape') {
    keyEvent.preventDefault();
    person.name = editOriginalName;
    editingGroupId = null;
    editingPersonId = null;
    editOriginalName = '';
    renderEventScreen();
  }
});
input.addEventListener('blur', () => {
  commitPersonEdit(event, group, person, input.value);
});
main.append(input);
setTimeout(() => {
  input.focus();
  input.setSelectionRange(input.value.length, input.value.length);
}, 0);
} else {
  const line = document.createElement('div');
  line.className = 'person-name-line';
  const name = document.createElement('div');
  name.className = 'person-name';
  name.textContent = person.name;
  line.append(name);
  if (person.walkin) {
    const badge = document.createElement('span');
    badge.className = 'walkin-badge';
    badge.textContent = 'walk-in';
    line.append(badge);
  }
  main.append(line);
}

const actions = document.createElement('div');
actions.className = 'person-actions';
if (pendingPersonRemoveId === `${group.id}:${person.id}`) {
  const confirm = document.createElement('button');
  confirm.className = 'small-btn red';
  confirm.type = 'button';
  confirm.textContent = 'Confirm remove';
  confirm.addEventListener('click', () => {
    group.people = group.people.filter(item => item.id !== person.id);
    pendingPersonRemoveId = null;
    touchEvent(event);
    renderEventScreen();
    if (currentSheet === 'search') renderSearchResults();
  });
  const cancel = document.createElement('button');
  cancel.className = 'small-btn';
  cancel.type = 'button';
  cancel.textContent = 'Cancel';
  cancel.addEventListener('click', () => {
    pendingPersonRemoveId = null;
    renderEventScreen();
  });
}

```

```

});
actions.append(confirm, cancel);
} else {
  const edit = document.createElement('button');
  edit.className = 'small-btn';
  edit.type = 'button';
  edit.textContent = 'Edit';
  edit.addEventListener('click', () => {
    editingGroupId = group.id;
    editingPersonId = person.id;
    editOriginalName = person.name;
    renderEventScreen();
  });
  const remove = document.createElement('button');
  remove.className = 'small-btn red';
  remove.type = 'button';
  remove.textContent = 'Remove';
  remove.addEventListener('click', () => {
    pendingPersonRemoveId = `${group.id}:${person.id}`;
    pendingGroupDeleteId = null;
    renderEventScreen();
  });
  actions.append(edit, remove);
}

row.append(avatar, main, actions);
return row;
}

function commitPersonEdit(event, group, person, value) {
  if (editingGroupId !== group.id || editingPersonId !== person.id) return;
  const normalized = normalizeName(value);
  if (!normalized) {
    person.name = editOriginalName;
  } else {
    person.name = normalized;
    if (findAlphaMatch(normalized, event.alphaList)) {
      person.walkin = false;
      person.note = '';
    }
  }
  editingGroupId = null;
  editingPersonId = null;
  editOriginalName = '';
  touchEvent(event);
  renderEventScreen();
}

function createNewGroup() {
  const event = currentEvent();
  if (!event) return;
  const next = event.groups.reduce((max, group) => Math.max(max, group.number),
0) + 1;
  const group = {
    id: uid(),
    number: next,
    label: `Photo Group ${next}`,
    people: []
  };
  event.groups.push(group);
  touchEvent(event);
  renderEventScreen();
}

```

```

    setTimeout(() => {
      const card = `${group-${group.id}`);
      if (card) card.scrollIntoView({ behavior: 'smooth', block: 'center' });
      openSearchSheet(group.id);
    }, 90);
  }

function openExportScreen() {
  const event = currentEvent();
  if (!event) return;
  if (!event.groups.length) {
    showToast('No groups to export yet');
    return;
  }
  renderExportScreen();
  push('s-export');
}

function renderExportScreen() {
  $('export-preview').textContent = buildExport();
}

function buildExport() {
  const event = currentEvent();
  if (!event) return 'Photo #\tFirst / Last Name:\tNotes: ';
  const rows = ['Photo #\tFirst / Last Name:\tNotes: '];
  event.groups
    .forEach(group => {
      if (!group.people.length) {
        rows.push(`${group.number}\t(empty group)\t`);
        return;
      }
      group.people.forEach((person, index) => {
        const photo = index === 0 ? String(group.number) : '';
        const note = person.walkin ? 'Not on Alpha List' : '';
        rows.push(`${photo}\t${person.name}\t${note}`);
      });
    });
  return rows.join('\n');
}

async function copyExport() {
  const text = buildExport();
  try {
    if (navigator.clipboard && typeof navigator.clipboard.writeText === 'function') {
      try {
        await navigator.clipboard.writeText(text);
        showToast('Copied');
        return;
      } catch (clipboardError) {
        fallbackCopy(text);
        showToast('Copied');
        return;
      }
    }
    fallbackCopy(text);
    showToast('Copied');
  } catch (error) {
    showToast('Copy failed - select and copy the text manually');
  }
}

```

```

function fallbackCopy(text) {
  const textarea = document.createElement('textarea');
  textarea.value = text;
  textarea.setAttribute('readonly', '');
  textarea.style.position = 'fixed';
  textarea.style.left = '-9999px';
  textarea.style.top = '0';
  document.body.append(textarea);
  textarea.select();
  const ok = document.execCommand('copy');
  textarea.remove();
  if (!ok) throw new Error('Copy command failed');
}

function downloadExport() {
  try {
    const event = currentEvent();
    if (!event) return;
    const text = buildExport();
    const blob = new Blob([text], { type: 'text/plain;charset=utf-8' });
    const url = URL.createObjectURL(blob);
    const safeName = sanitizeFilename(event.name);
    const a = document.createElement('a');
    a.href = url;
    a.download = `LOOK_${safeName}_${event.date || todayISO()}.txt`;
    document.body.append(a);
    a.click();
    a.remove();
    setTimeout(() => URL.revokeObjectURL(url), 1000);
    showToast('Download started');
  } catch (error) {
    showToast('Download failed');
  }
}

function sanitizeFilename(name) {
  return (name || 'Event').replace(/^[^A-Za-z0-9]+/g, '_').replace(/^\_+|_\+$/g, '')
  || 'Event';
}

function openSheet(name) {
  const overlay = $('sheet-overlay');
  const sheet = $('sheet-${name}');
  currentSheet = name;
  document.body.classList.add('sheet-open');
  overlay.hidden = false;
  sheet.hidden = false;
  requestAnimationFrame(() => {
    overlay.classList.add('open');
    sheet.classList.add('open');
  });
  setTimeout(() => focusFirstSheetControl(name), 420);
}

function closeSheet() {
  if (!currentSheet) return;
  const closing = currentSheet;
  if (closing === 'ocr' && ocrBusy) {
    ocrCancelled = true;
    terminateOCRWorker('OCR stopped');
  }
}

```

```

const overlay = $('sheet-overlay');
const sheet = $('sheet-${closing}');
overlay.classList.remove('open');
sheet.classList.remove('open');
setTimeout(() => {
  if (currentSheet === closing) {
    overlay.hidden = true;
    sheet.hidden = true;
    document.body.classList.remove('sheet-open');
    currentSheet = null;
    if (closing === 'search') {
      searchGroupId = null;
      $('person-search').value = '';
    }
    if (closing === 'paste') {
      $('paste-text').value = '';
    }
    if (closing === 'ocr') {
      resetOCRState();
    }
  }
}, 260);
}

function focusFirstSheetControl(name) {
  if (name === 'search') $('person-search').focus();
  if (name === 'paste') $('paste-text').focus();
  if (name === 'ocr' && !ocrBusy) {
    const input = $('ocr-list').querySelector('input');
    if (input) input.focus();
  }
}

function openSearchSheet(groupId) {
  searchGroupId = groupId;
  $('person-search').value = '';
  renderSearchSheet();
  openSheet('search');
}

function renderSearchSheet() {
  const event = currentEvent();
  const group = event ? event.groups.find(item => item.id === searchGroupId) : null;
  $('search-title').textContent = group ? `Add to ${group.label}` : 'Add to Group';
  renderSearchResults();
}

function renderSearchResults() {
  const event = currentEvent();
  const group = event ? event.groups.find(item => item.id === searchGroupId) : null;
  const container = $('search-results');
  const query = cleanQuery($('person-search').value);
  container.textContent = '';
  if (!event || !group) return;

  const results = getSearchResults(query, event.alphaList, group);
  results.forEach(result => {
    const row = document.createElement('button');
    row.type = 'button';
  });
}

```



```

    row.className = `result-row ${result.already ? '' : 'can-add'}`;
    if (result.already) {
        row.setAttribute('aria-disabled', 'true');
    } else {
        row.addEventListener('click', () => addPersonToGroup(group.id, result.entry.display, false, ''));
    }

    const avatar = document.createElement('div');
    avatar.className = 'avatar';
    avatar.textContent = initials(result.entry.display);

    const main = document.createElement('div');
    const name = document.createElement('div');
    name.className = 'result-name';
    name.textContent = result.entry.display;
    const sub = document.createElement('div');
    sub.className = 'result-sub';
    sub.textContent = 'Alpha list';
    main.append(name, sub);

    const action = document.createElement('div');
    if (result.already) {
        action.className = 'in-group';
        action.textContent = 'In group';
    } else {
        const plus = document.createElement('span');
        plus.className = 'plus-btn';
        plus.setAttribute('aria-hidden', 'true');
        plus.textContent = '+';
        action.append(plus);
    }
    row.append(avatar, main, action);
    container.append(row);
});

if (query) {
    const walkin = document.createElement('button');
    walkin.type = 'button';
    walkin.className = 'walkin-row';
    walkin.addEventListener('click', () => addPersonToGroup(group.id, query, true, 'Not on Alpha List'));

    const avatar = document.createElement('div');
    avatar.className = 'avatar alt-3';
    avatar.textContent = initials(query);
    const main = document.createElement('div');
    const name = document.createElement('div');
    name.className = 'result-name';
    name.textContent = `Add "${query}" as walk-in`;
    const sub = document.createElement('div');
    sub.className = 'result-sub';
    sub.textContent = 'not on alpha list';
    main.append(name, sub);
    const plus = document.createElement('span');
    plus.className = 'plus-btn';
    plus.setAttribute('aria-hidden', 'true');
    plus.textContent = '+';
    walkin.append(avatar, main, plus);
    container.append(walkin);
} else {
    const empty = document.createElement('div');

```

```

    empty.className = 'empty';
    empty.textContent = 'Start typing a guest name.';
    container.append(empty);
  }
}

function handleSearchKeydown(event) {
  if (event.key !== 'Enter') return;
  event.preventDefault();
  const activeEvent = currentEvent();
  const group = activeEvent ? activeEvent.groups.find(item => item.id === search
GroupId) : null;
  const query = cleanQuery($('person-search').value);
  if (!activeEvent || !group || !query) return;
  const results = getSearchResults(query, activeEvent.alphaList, group);
  const strong = results.filter(result => result.score >= 90);
  if (strong.length === 1 && !strong[0].already) {
    addPersonToGroup(group.id, strong[0].entry.display, false, '');
    return;
  }
  if (strong.length === 0) {
    addPersonToGroup(group.id, query, true, 'Not on Alpha List');
  }
}

function addPersonToGroup(groupId, rawName, walkin, note) {
  const event = currentEvent();
  const group = event ? event.groups.find(item => item.id === groupId) : null;
  const name = normalizeName(rawName);
  if (!event || !group || !name) return;
  const exists = group.people.some(person => person.name.toLowerCase() === name.
toLowerCase());
  if (exists) {
    showToast('Already in group');
    $('person-search').value = '';
    renderSearchResults();
    setTimeout(() => $('person-search').focus(), 0);
    return;
  }
  group.people.push({
    id: uid(),
    name,
    walkin,
    note: walkin ? 'Not on Alpha List' : note,
    addedAt: Date.now()
  });
  touchEvent(event);
  renderEventScreen();
  renderSearchResults();
  showToast(`${name} added`);
  $('person-search').value = '';
  renderSearchResults();
  setTimeout(() => $('person-search').focus(), 0);
}

function getSearchResults(query, alphaList, group) {
  if (!query) return [];
  return alphaList
    .map(entry => {
      const score = scoreAlphaEntry(query, entry);
      const already = group.people.some(person => person.name.toLowerCase() ===
entry.display.toLowerCase());

```

```

        return { entry, score, already };
    })
    .filter(result => result.score > 0)
    .sort((a, b) => {
        if (b.score !== a.score) return b.score - a.score;
        const last = a.entry.last.localeCompare(b.entry.last, undefined, { sensitivity: 'base' });
        if (last !== 0) return last;
        return a.entry.first.localeCompare(b.entry.first, undefined, { sensitivity: 'base' });
    })
    .slice(0, 25);
}

function scoreAlphaEntry(query, entry) {
    const q = cleanQuery(query).toLowerCase();
    const full = cleanQuery(entry.display).toLowerCase();
    const first = cleanQuery(entry.first).toLowerCase();
    const last = cleanQuery(entry.last).toLowerCase();
    if (full === q) return 100;
    if (last === q) return 95;
    if (last.startsWith(q)) return 90;
    if (full.includes(q) || first.includes(q)) return 75;
    if (q.length >= 3 && levenshtein(q, last) <= 1) return 55;
    return 0;
}

function levenshtein(a, b) {
    const dp = Array.from({ length: a.length + 1 }, () => new Array(b.length + 1).fill(0));
    for (let i = 0; i <= a.length; i += 1) dp[i][0] = i;
    for (let j = 0; j <= b.length; j += 1) dp[0][j] = j;
    for (let i = 1; i <= a.length; i += 1) {
        for (let j = 1; j <= b.length; j += 1) {
            const cost = a[i - 1] === b[j - 1] ? 0 : 1;
            dp[i][j] = Math.min(dp[i - 1][j] + 1, dp[i][j - 1] + 1, dp[i - 1][j - 1] + cost);
        }
    }
    return dp[a.length][b.length];
}

function openPasteSheet() {
    $('paste-text').value = '';
    openSheet('paste');
}

function addPastedNames() {
    try {
        const parsed = parseAlphaText($('paste-text').value);
        const existing = new Set(setupAlphaList.map(name => name.display.toLowerCase()));
        const additions = parsed.filter(name => !existing.has(name.display.toLowerCase()));
        setSetupAlphaList(setupAlphaList.concat(additions));
        closeSheet();
        showToast(`Added ${additions.length} ${additions.length === 1 ? 'name' : 'names'}` ($ {setupAlphaList.length} total));
    } catch (error) {
        showToast('Could not add names');
    }
}

```

```

function handleImageInput(input) {
  const file = input.files && input.files[0];
  input.value = '';
  if (!file) return;
  startOCR(file);
}

function handleFileInput(input) {
  const file = input.files && input.files[0];
  input.value = '';
  if (!file) return;
  importFile(file);
}

async function importFile(file) {
  clearSetupError();
  try {
    const name = file.name.toLowerCase();
    if (name.endsWith('.csv') || name.endsWith('.txt')) {
      const text = await readFileAsText(file);
      setSetupAlphaList(parseAlphaText(text));
      showToast(`${setupAlphaList.length} ${setupAlphaList.length === 1 ? 'name' : 'names'} loaded`);
      return;
    }
    const names = await parseExcelFile(file);
    setSetupAlphaList(names);
    showToast(`${setupAlphaList.length} ${setupAlphaList.length === 1 ? 'name' : 'names'} loaded`);
  } catch (error) {
    showSetupError('Could not import file. Try CSV or paste instead.');
```

```

let firstIndex = 0;
let lastIndex = 1;
let startRow = 0;
const maxHeaderRows = Math.min(5, rows.length);
for (let r = 0; r < maxHeaderRows; r += 1) {
  const cells = rows[r].map(cell => String(cell || '').trim());
  const firstCol = cells.findIndex(cell => /\bfirst\b/i.test(cell));
  const lastCol = cells.findIndex(cell => /\blast\b/i.test(cell));
  if (firstCol >= 0 && lastCol >= 0) {
    firstIndex = firstCol;
    lastIndex = lastCol;
    startRow = r + 1;
    break;
  }
}
const names = [];
for (let r = startRow; r < rows.length; r += 1) {
  const row = rows[r];
  const first = stripQuotes(String(row[firstIndex] || '').trim());
  const last = stripQuotes(String(row[lastIndex] || '').trim());
  if (!first && !last) continue;
  const display = normalizeName(`${first} ${last}`);
  if (display) names.push(nameFromDisplay(display));
}
return sortAndDedupeNames(names);
}

function stripQuotes(value) {
  return value.replace(/^[\'"]+|[\\'"]+$/g, '');
}

function startOCR(file) {
  openOCRSheet();
  runOCR(file, ocrSession);
}

function openOCRSheet() {
  ocrSession += 1;
  ocrBusy = true;
  ocrCancelled = false;
  ocrNames = [];
  $('ocr-title').textContent = 'Reading image...';
  $('ocr-processing').hidden = false;
  $('ocr-progress').textContent = 'Loading OCR engine...';
  $('ocr-review').hidden = true;
  $('ocr-list').textContent = '';
  $('ocr-use').disabled = true;
  $('ocr-error').textContent = '';
  $('ocr-error').classList.remove('visible');
  openSheet('ocr');
}

async function runOCR(file, session) {
  let worker = null;
  try {
    if (!window.Tesseract) throw new Error('Tesseract unavailable');
    const blob = await preprocessImageForOCR(file);
    if (ocrCancelled || session !== ocrSession) return;
    $('ocr-progress').textContent = 'Loading OCR engine...';
    worker = await window.Tesseract.createWorker({
      logger: message => updateOCRProgress(message)
    });
  }
}

```

```

activeOCRWorker = worker;
if (ocrCancelled || session !== ocrSession) {
  await worker.terminate();
  activeOCRWorker = null;
  worker = null;
  return;
}
await worker.loadLanguage('eng');
if (ocrCancelled || session !== ocrSession) return;
await worker.initialize('eng');
if (ocrCancelled || session !== ocrSession) return;
await worker.setParameters({
  tessedit_pageseg_mode: window.Tesseract.PSM.AUTO,
  tessedit_char_whitelist: "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz",
  preserve_interword_spaces: '1',
  user_defined_dpi: '300'
});
if (ocrCancelled || session !== ocrSession) return;
const imageUrl = URL.createObjectURL(blob);
let text = '';
try {
  const result = await worker.recognize(imageUrl);
  text = result && result.data ? result.data.text : '';
} finally {
  URL.revokeObjectURL(imageUrl);
}
if (activeOCRWorker === worker) {
  await worker.terminate();
  activeOCRWorker = null;
}
worker = null;
if (ocrCancelled || session !== ocrSession) return;
ocrBusy = false;
ocrNames = parseAlphaText(text);
renderOCRReview();
} catch (error) {
  if (worker && activeOCRWorker === worker) {
    try {
      await worker.terminate();
    } catch (terminateError) {
      showToast('OCR cleanup failed');
    }
  }
  activeOCRWorker = null;
  ocrBusy = false;
  if (!ocrCancelled && session === ocrSession) showOCRError();
}
}

function updateOCRProgress(message) {
  if (!currentSheet || currentSheet !== 'ocr') return;
  if (message.status === 'recognizing text') {
    const pct = Math.max(0, Math.min(100, Math.round((message.progress || 0) * 100)));
    $('ocr-progress').textContent = `Reading... ${pct}%`;
  } else if (message.status) {
    $('ocr-progress').textContent = message.status.charAt(0).toUpperCase() + message.status.slice(1) + '...';
  }
}

```

```

function showOCRError() {
  $('ocr-title').textContent = 'Reading failed';
  $('ocr-processing').hidden = true;
  $('ocr-review').hidden = true;
  const err = $('ocr-error');
  err.textContent = 'Could not read image. Try the paste option instead.';
  err.classList.add('visible');
}

function renderOCRReview() {
  $('ocr-title').textContent = `Found ${ocrNames.length} ${ocrNames.length === 1
? 'name' : 'names'} - review & edit`;
  $('ocr-processing').hidden = true;
  $('ocr-review').hidden = false;
  renderOCRList();
  setTimeout(() => focusFirstSheetControl('ocr'), 0);
}

function renderOCRList() {
  const list = $('ocr-list');
  list.textContent = '';
  ocrNames.forEach((nameObj, index) => {
    const row = document.createElement('div');
    row.className = 'ocr-row';
    const input = document.createElement('input');
    input.className = 'name-input';
    input.type = 'text';
    input.value = nameObj.display;
    input.setAttribute('aria-label', `OCR name ${index + 1}`);
    input.addEventListener('input', () => {
      ocrNames[index] = nameFromDisplay(input.value);
      updateOCRUseButton();
    });
    const remove = document.createElement('button');
    remove.className = 'small-btn red';
    remove.type = 'button';
    remove.textContent = 'Remove';
    remove.addEventListener('click', () => {
      ocrNames.splice(index, 1);
      renderOCRList();
    });
    row.append(input, remove);
    list.append(row);
  });
  updateOCRUseButton();
}

function updateOCRUseButton() {
  const usable = Array.from($('ocr-list').querySelectorAll('input')).map(input =>
  > input.value.trim()).filter(Boolean);
  $('ocr-use').disabled = ocrBusy || usable.length < 1;
}

function useOCRNames() {
  try {
    const values = Array.from($('ocr-list').querySelectorAll('input')).map(input
    => input.value.trim()).filter(Boolean);
    const names = sortAndDedupeNames(values.map(value => nameFromDisplay(normali
    zeName(value))).filter(name => name.display));
    if (!names.length) {
      updateOCRUseButton();
      return;
    }
  }
}

```

```

    }
    setSetupAlphaList(names);
    closeSheet();
    showToast(`${setupAlphaList.length} ${setupAlphaList.length === 1 ? 'name' :
'names'}} loaded`);
  } catch (error) {
    showToast('Could not use OCR names');
  }
}

function terminateOCRWorker(message) {
  ocrSession += 1;
  const worker = activeOCRWorker;
  activeOCRWorker = null;
  ocrBusy = false;
  if (worker) {
    worker.terminate()
      .then(() => {
        if (message) showToast(message);
      })
      .catch(() => showToast('OCR cleanup failed'));
  }
}

function resetOCRState() {
  activeOCRWorker = null;
  ocrBusy = false;
  ocrCancelled = false;
  ocrNames = [];
  $('ocr-list').textContent = '';
}

async function preprocessImageForOCR(file) {
  const sourceCanvas = await loadImageCanvas(file);
  const thresholdCanvas = adaptiveThresholdCanvas(sourceCanvas);
  const finalCanvas = upscaleCanvasIfNeeded(thresholdCanvas, 2550);
  return await new Promise((resolve, reject) => {
    finalCanvas.toBlob(blob => {
      if (blob) resolve(blob);
      else reject(new Error('Canvas export failed'));
    }, 'image/png');
  });
}

async function loadImageCanvas(file) {
  if ('createImageBitmap' in window) {
    try {
      const bitmap = await createImageBitmap(file);
      const canvas = document.createElement('canvas');
      canvas.width = bitmap.width;
      canvas.height = bitmap.height;
      const ctx = canvas.getContext('2d', { willReadFrequently: true });
      ctx.drawImage(bitmap, 0, 0);
      bitmap.close();
      return canvas;
    } catch (error) {
      showToast('Using alternate image reader');
    }
  }
  return await loadImageCanvasFallback(file);
}

```



```

function loadImageCanvasFallback(file) {
  return new Promise((resolve, reject) => {
    const url = URL.createObjectURL(file);
    const image = new Image();
    image.onload = () => {
      URL.revokeObjectURL(url);
      const canvas = document.createElement('canvas');
      canvas.width = image.naturalWidth;
      canvas.height = image.naturalHeight;
      const ctx = canvas.getContext('2d', { willReadFrequently: true });
      ctx.drawImage(image, 0, 0);
      resolve(canvas);
    };
    image.onerror = () => {
      URL.revokeObjectURL(url);
      reject(new Error('Image load failed'));
    };
    image.src = url;
  });
}

function adaptiveThresholdCanvas(canvas) {
  const w = canvas.width;
  const h = canvas.height;
  const ctx = canvas.getContext('2d', { willReadFrequently: true });
  const img = ctx.getImageData(0, 0, w, h);
  const data = img.data;
  const gray = new Uint8ClampedArray(w * h);
  for (let i = 0, p = 0; i < data.length; i += 4, p += 1) {
    gray[p] = Math.round(0.299 * data[i] + 0.587 * data[i + 1] + 0.114 * data[i + 2]);
  }

  const integral = new Float64Array((w + 1) * (h + 1));
  for (let y = 1; y <= h; y += 1) {
    let rowSum = 0;
    for (let x = 1; x <= w; x += 1) {
      rowSum += gray[(y - 1) * w + (x - 1)];
      integral[y * (w + 1) + x] = integral[(y - 1) * (w + 1) + x] + rowSum;
    }
  }

  const half = 16;
  for (let y = 0; y < h; y += 1) {
    const y1 = Math.max(0, y - half);
    const y2 = Math.min(h - 1, y + half);
    for (let x = 0; x < w; x += 1) {
      const x1 = Math.max(0, x - half);
      const x2 = Math.min(w - 1, x + half);
      const area = (x2 - x1 + 1) * (y2 - y1 + 1);
      const sum = rectSum(integral, w + 1, x1, y1, x2, y2);
      const avg = sum / area;
      const out = gray[y * w + x] < avg - 10 ? 0 : 255;
      const i = (y * w + x) * 4;
      data[i] = out;
      data[i + 1] = out;
      data[i + 2] = out;
      data[i + 3] = 255;
    }
  }
  ctx.putImageData(img, 0, 0);
  return canvas;
}

```

```

}

function rectSum(integral, stride, x1, y1, x2, y2) {
  const ax = x1;
  const ay = y1;
  const bx = x2 + 1;
  const by = y2 + 1;
  return integral[by * stride + bx] - integral[ay * stride + bx] - integral[by *
stride + ax] + integral[ay * stride + ax];
}

function upscaleCanvasIfNeeded(canvas, targetWidth) {
  if (canvas.width >= targetWidth) return canvas;
  const scale = targetWidth / canvas.width;
  const out = document.createElement('canvas');
  out.width = Math.round(canvas.width * scale);
  out.height = Math.round(canvas.height * scale);
  const ctx = out.getContext('2d');
  ctx.imageSmoothingEnabled = false;
  ctx.drawImage(canvas, 0, 0, out.width, out.height);
  return out;
}

function parseAlphaText(rawText) {
  const headers = /^(First|Last|Name|Guest|Guests|Group|Table|Page|Alpha List|At
tendeelAttendees)$/i;
  const names = [];
  String(rawText || '').
    .split(/\r?\n/)
    .forEach(rowLine => {
      let line = rowLine.trim();
      if (!line || line.length < 4) return;
      if (/^\d+$/.test(line)) return;
      if (headers.test(line)) return;
      let previous = '';
      while (previous !== line) {
        previous = line;
        line = line.replace(/\s*(?:Table\s+\d+\s*|Tbl\s+\d+|T-\d+|#\d+|\s*)\s
*$/i, '').trim();
      }
      line = line.replace(/\s+/g, ' ');
      if (!line || !/^[A-Z]/.test(line)) return;
      if (!/^[A-Za-z'\s-]+\s$/i.test(line)) return;
      const parts = line.split(' ').filter(Boolean);
      if (parts.length < 2 || parts.length > 5) return;
      const first = normalizeName(parts[0]);
      const last = normalizeName(parts.slice(1).join(' '));
      if (!first || !last) return;
      names.push({ first, last, display: `${first} ${last}` });
    });
  return sortAndDedupeNames(names);
}

function normalizeName(str) {
  return String(str || '').
    .trim()
    .replace(/\s+/g, ' ')
    .split(' ')
    .map(word => word
      .toLowerCase()
      .replace(/(^|'[-.])([a-z])/g, (match, prefix, letter) => prefix + letter.
toUpperCase()))

```

```

        .join(' ')
        .trim();
    }

    function findAlphaMatch(name, alphaList) {
        const target = normalizeName(name).toLowerCase();
        return alphaList.find(entry => entry.display.toLowerCase() === target) || null;
    }

    function initials(name) {
        const parts = normalizeName(name).split(' ').filter(Boolean);
        if (!parts.length) return '';
        if (parts.length === 1) return parts[0].charAt(0).toUpperCase();
        return (parts[0].charAt(0) + parts[parts.length - 1].charAt(0)).toUpperCase();
    }

    function nameFromDisplay(display) {
        const normalized = normalizeName(display);
        const parts = normalized.split(' ').filter(Boolean);
        if (parts.length < 2) return { first: normalized, last: '', display: normalized };
        return {
            first: parts[0],
            last: parts.slice(1).join(' '),
            display: normalized
        };
    }

    function sortAndDedupeNames(names) {
        const seen = new Set();
        const clean = [];
        names.forEach(name => {
            const obj = typeof name === 'string' ? nameFromDisplay(name) : name;
            const display = normalizeName(obj.display || `${obj.first} ${obj.last}`);
            if (!display) return;
            const normalizedObj = nameFromDisplay(display);
            const key = normalizedObj.display.toLowerCase();
            if (seen.has(key)) return;
            seen.add(key);
            clean.push(normalizedObj);
        });
        return clean.sort((a, b) => {
            const last = a.last.localeCompare(b.last, undefined, { sensitivity: 'base' });
            if (last !== 0) return last;
            return a.first.localeCompare(b.first, undefined, { sensitivity: 'base' });
        });
    }

    function cleanQuery(value) {
        return normalizeName(String(value || '').replace(/\s+/g, ' '));
    }

    if ('serviceWorker' in navigator) {
        window.addEventListener('load', () => {
            navigator.serviceWorker.register('./sw.js', { scope: './' })
                .catch(() => {
                    showToast('Offline setup could not be completed');
                });
        });
    }

```

```
}  
</script>  
</body>  
</html>
```